a copy rather than a reference (so you're following the guideline from section 3.2.2) and protect the internal data with a mutex, this interface is still inherently subject to race conditions. This problem is not unique to a mutex-based implementation; it's an interface problem, so the race conditions would still occur with a lock-free implementation.

Listing 3.3. The interface to the `std::stack` container adapter

```
template<typename T,typename Container=std::deque<T> >
class stack
{
public:
    explicit stack(const Container&);
    explicit stack(Container&& = Container());
    template <class Alloc> explicit stack(const Alloc&);
    template <class Alloc> stack(const Container&, const Alloc&);
    template <class Alloc> stack(Container&&, const Alloc&);
    template <class Alloc> stack(stack&&, const Alloc&);

    bool empty() const;
    size_t size() const;
    T& top();
    T const& top() const;
    void push(T const&);
    void push(T&&);
    void pop();
    void swap(stack&&);
};
```

The problem here is that the results of `empty()` and `size()` can't be relied on. Although they might be correct at the time of the call, once they've returned, other threads are free to access the stack and might `push()` new elements onto or `pop()` the existing ones off of the stack before the thread that called `empty()` or `size()` could use that information.

In particular, if the `stack` instance is *not shared*, it's safe to check for `empty()` and then call `top()` to access the top element if the stack is not empty, as follows: